

SOLID WALL CONSULTING

Agilité, DevOps & Sécurité

Le cours expliqué pas à pas, en images

Formateur : Haithem Mihoubi

Comment lire cette présentation

On avance **pas à pas** : pour chaque notion, le problème, l'explication, et un schéma.

- 3 modules : **Agilité**, **DevOps**, **Spring Security**
- Un fil rouge unique : l'application **QuickBite**

Chaque slide = une seule idée, illustrée



MODULE 1

Agilité

Scrum & Kanban

Le problème de départ

Avant l'Agilité, on faisait tout en **cascade** : spécifier, développer, puis livrer 12 à 18 mois plus tard.

- Effet tunnel : le client ne voit rien avant la fin
- Les besoins ont changé entre-temps → produit périmé
- Le coût du changement explose avec le temps

Cascade : la valeur arrive à la fin



Agile : de la valeur à chaque itération



L'idée de l'Agilité

Si on ne peut pas tout prévoir, organisons-nous pour **apprendre et corriger vite**.

- Livrer un **petit morceau utilisable** toutes les 2-3 semaines
- Montrer à de vrais utilisateurs, recueillir leur avis, ajuster

Un grand pari risqué devient une série de petits paris corrigibles

Cascade : la valeur arrive à la fin



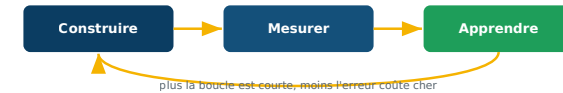
Agile : de la valeur à chaque itération



Le Manifeste Agile (2001)

Quatre **valeurs** : « A plutôt que B » — B garde de la valeur, mais on privilégie A.

- Les individus et interactions > les processus et outils
- Un logiciel qui marche > une documentation exhaustive
- La collaboration client > la négociation de contrat
- L'adaptation au changement > le suivi d'un plan

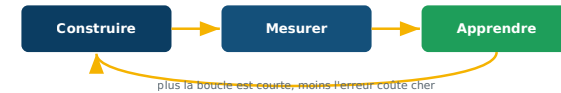


La boucle de feedback

Le cœur de l'Agilité : **construire** → **mesurer** → **apprendre**, en boucle.

- Plus la boucle est courte, plus on corrige tôt
- Corriger tôt coûte beaucoup moins cher

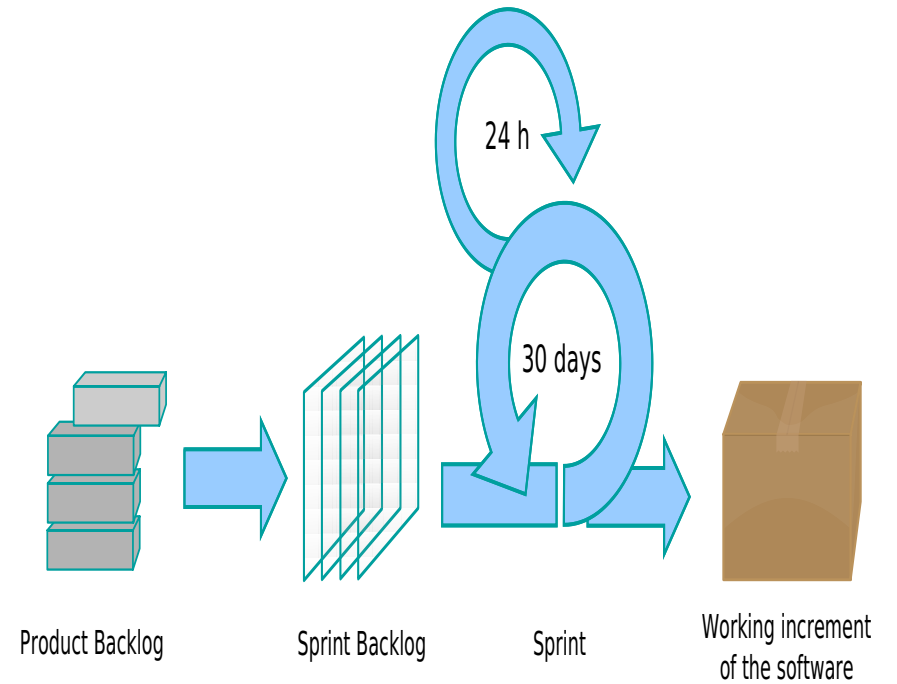
MVP : la plus petite version qui permet déjà d'apprendre



Scrum : le cadre

Scrum organise le travail en **sprints** (itérations de durée fixe, 1 à 4 semaines).

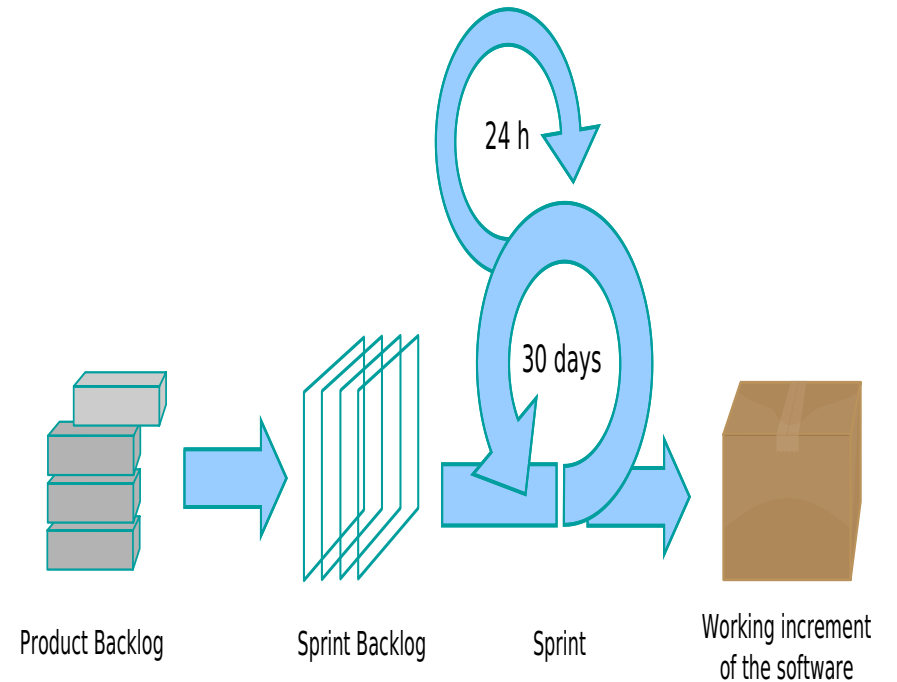
- 3 piliers : transparence, inspection, adaptation
- Événements : Planning, Daily, Review, Rétrospective
- Artefacts : Product Backlog, Sprint Backlog, Increment



Scrum : les 3 rôles

Une seule équipe, sans hiérarchie, avec trois responsabilités.

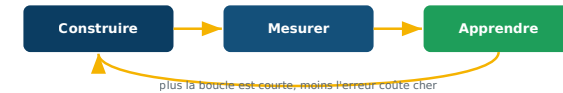
- **Product Owner** : le quoi et le pourquoi (priorise la valeur)
- **Scrum Master** : facilite, lève les obstacles (≠ chef de projet)
- **Développeurs** : auto-organisés, le comment



User stories & estimation

On décrit le besoin côté utilisateur : « En tant que... je veux... afin de... ».

- Critères **INVEST** pour une bonne story
- Estimation en **points** (Fibonacci), pas en heures
- Priorisation **MoSCoW** : Must / Should / Could / Won't



Kanban : visualiser & limiter le travail

On visualise le flux sur un tableau et on **limite le travail en cours (WIP)** — « stop starting, start finishing ». Loi de Little : $\text{Lead time} \approx \text{WIP} \div \text{Throughput}$.



MODULE 2

DevOps

CI/CD · Docker · Kubernetes

DevOps : briser le mur

Réunir **Dev** (le changement) et **Ops** (la stabilité) par la culture et l'automatisation.

- **CAMS** : Culture, Automation, Measurement, Sharing
- But : raccourcir le délai idée → production, de façon fiable

Métriques DORA : fréquence, délai, taux d'échec, MTTR



Intégration & livraison continues

À chaque commit, un **pipeline** teste, construit et prépare le déploiement.

- **CI** : tester à chaque changement
- **CD** : toujours déployable (puis déploiement auto)
- Règle d'or : on ne fusionne jamais un pipeline rouge

À chaque push : tout est automatisé



✗ tests rouges → fusion bloquée

Git : le travail collaboratif

Tout part du code, géré avec **Git** : branches courtes
+ Pull Requests + revue.

- Conventional Commits : feat, fix, docs, refactor...
- GitHub / GitLab : pipelines intégrés

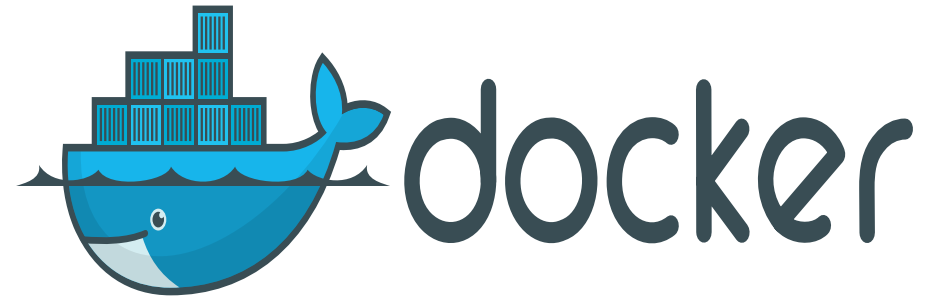
Branches courtes = fusions faciles



Docker : la conteneurisation

Empaqueter l'application **avec son environnement**
→ « ça marche partout ».

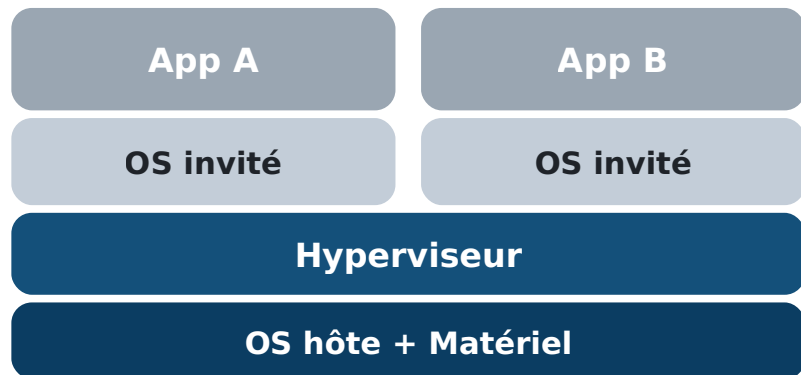
- Image (le modèle) vs Conteneur (l'instance)
- Dockerfile multi-stage → image légère
- Docker Compose : plusieurs services en une commande



Conteneur vs machine virtuelle

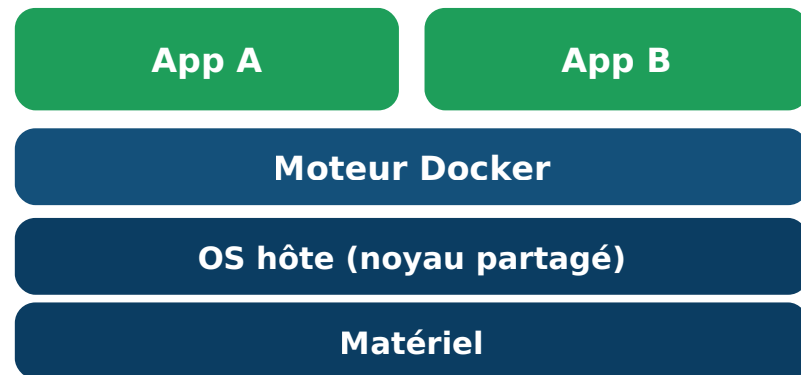
Le conteneur **partage le noyau** de l'hôte : léger, démarre en millisecondes. La VM embarque un OS complet : lourde, démarre en minutes.

Machines virtuelles



lourd · démarre en minutes

Conteneurs



léger · démarre en millisecondes

Kubernetes : l'orchestration

En production : redémarrage auto, mise à l'échelle, déploiement **sans coupure**.

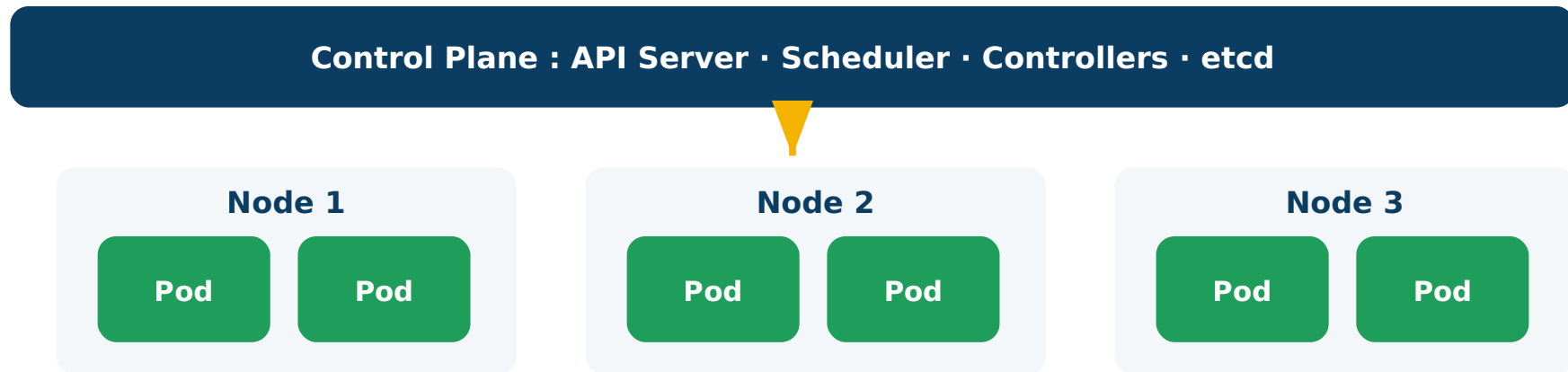
- **Pod** (unité), **Deployment** (réplicas), **Service** (adresse stable)
- **Ingress** (accès externe), ConfigMap / Secret
- Déclaratif : on décrit l'état souhaité, K8s converge



Architecture Kubernetes

Un **Control Plane** (le cerveau) pilote des **Nodes** (machines) qui exécutent les Pods.

Architecture Kubernetes



Observabilité

Voir ce qui se passe en production : logs, **métriques**, traces.

- **Prometheus** collecte les métriques
- **Grafana** affiche les tableaux de bord
- 4 signaux dorés : latence, trafic, erreurs, saturation



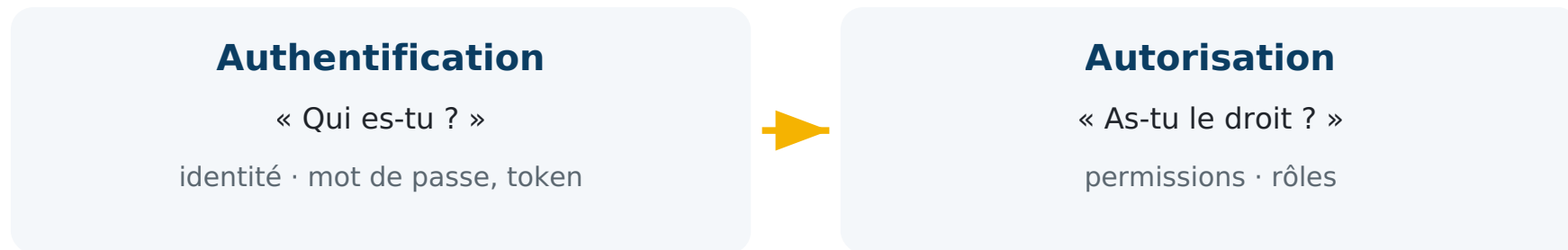
MODULE 3

Spring Security

JWT · OAuth2 · Keycloak

Authentification vs Autorisation

AuthN = « qui es-tu ? » (vient en premier). **AuthZ** = « as-tu le droit ? ». 401 = non authentifié · 403 = authentifié mais sans droit.



Spring Security 6

La sécurité s'insère comme une **chaîne de filtres** devant l'application.

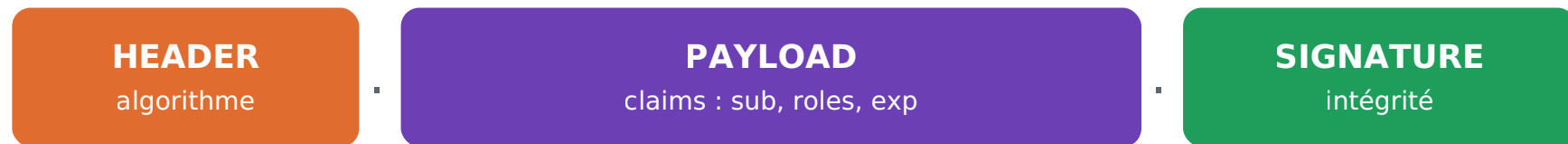
- Bean **SecurityFilterChain**
(WebSecurityConfigurerAdapter supprimé)
- Mots de passe : **BCrypt** / Argon2 (jamais en clair)
- RBAC : `@PreAuthorize("hasRole('ADMIN')")`



JWT : un jeton auto-porteur

Le JWT contient l'identité, est **signé** (donc vérifiable sans base) mais **pas chiffré** : son contenu est lisible.
Access token court + refresh token long avec rotation.

Un JWT = 3 parties (signé, pas chiffré)



⚠ le payload est lisible : aucune donnée sensible dedans

OAuth2 / OpenID Connect

OAuth2 délègue l'autorisation ; **OIDC** ajoute l'authentification (qui est l'utilisateur). Flow recommandé : Authorization Code + PKCE.

OAuth2 / OIDC — Authorization Code + PKCE

Utilisateur

Client

Auth Server

API

- 1 Utilisateur → Client : je veux me connecter
- 2 Client → Auth Server : redirige vers login
- 3 Auth Server → Client : code d'autorisation
- 4 Client → Auth Server : code + PKCE → tokens
- 5 Client → API : appel avec access token

Keycloak

Serveur d'identité (IAM) : l'API devient un **resource server** qui valide les tokens.

- Realm (espace isolé), Client, Roles, Mappers
- SSO : « se connecter avec... »

Mapper realm_access.roles → ROLE_* (cause n°1 des 403)



| Durcissement & OWASP

Sécuriser pour de vrai : on applique une **check-list**.

- CORS (permission) $\neq$ CSRF (attaque)
- Rate limiting, exceptions neutres, audit
- Risque n°1 OWASP : **Broken Access Control**



PROJET FIL ROUGE

QuickBite

On construit, déploie et sécurise une vraie API

QuickBite : atelier après atelier

Une API Spring Boot qui grandit en appliquant chaque notion du cours.

- Sécurité de base → RBAC → JWT → Keycloak → durcissement
- DevOps : Docker → CI → Kubernetes
- Tests fournis : collection **Postman**



Synthèse du cursus

Trois disciplines, un seul objectif : **livrer de la valeur, en confiance.**

- **Agile** : construire le bon produit
- **DevOps** : le livrer vite et sûrement
- **Sécurité** : protéger ce qui est livré



Merci !

Questions / Réponses — Haithem Mihoubi